Relazione Tecnica e Progettuale: ConversAI

Corso di Laurea in Ingegneria Elettronica · Insegnamento di Applicazioni Web, A.A. 2025/2026
Università degli Studi di Udine
Autore: Stefano Bortolussi · Matricola: 167602

1. Introduzione e obiettivi

ConversAI è un'applicazione web full-stack per gestire chat testuali con un assistente simulato, sul modello delle sezioni Projects di ChatGPT e dei workspace di Claude. Le interazioni sono organizzate su tre livelli: un progetto raccoglie più conversazioni, ciascuna dedicata a un topic, e ogni conversazione è una sequenza di messaggi scritti dall'utente o prodotti dal server. L'interfaccia permette di creare progetti e conversazioni, inviare prompt senza ricaricare la pagina e consultare lo storico con ricerca ed esportazione.

Il front-end è in HTML, CSS e JavaScript senza librerie, il back-end in Node.js con Express e la persistenza in SQLite. Oltre ai requisiti obbligatori (punti 4.1-4.4) sono state realizzate due funzionalità aggiuntive: la ricerca nello storico (4.5) e l'esportazione di una conversazione (4.7).

2. Architettura di sistema e stack tecnologico

L'applicazione segue il modello client-server a tre livelli. Il livello di presentazione gira nel browser e usa il DOM e la Fetch API per aggiornare la pagina in modo asincrono. Il livello applicativo, in Node.js con Express, gestisce routing, validazione, logica e simulazione delle risposte. Il livello dati è un file SQLite locale (`data/database.sqlite`). Il browser scarica una sola volta la pagina e poi scambia con il server soltanto JSON; HTTP non ha stato, perciò ogni richiesta contiene da sola quanto serve a servirla (l'identificativo della conversazione è nel path) e lo stato della sessione, cioè progetto e conversazione aperti, resta nel client.

Il codice è diviso in tre parti. `db.js` apre il database, crea le tabelle e ne esporta la connessione. `server.js` crea l'applicazione Express, registra i middleware, definisce gli endpoint e contiene la funzione di simulazione. La cartella `public/` contiene `index.html`, `style.css` e `app.js`. Un middleware è una funzione che intercetta ogni richiesta prima del gestore dell'endpoint: `express.json()` converte il corpo JSON in un oggetto disponibile in `req.body`, mentre `express.static` serve i file di `public/`.

Node.js esegue il JavaScript su un solo thread e gestisce l'I/O con un event loop: un'operazione lenta, come una query, registra una callback e restituisce subito il controllo, e il server può servire altre richieste nell'attesa. Per questo il codice che accede al database è annidato in callback e la risposta HTTP viene inviata al loro interno.

Stack: Node.js e npm; Express.js ^4.19.2; driver `sqlite3` ^5.1.7; nel client HTML5, CSS3 con variabili e JavaScript ES6+ con `async/await`, caricato come script classico (non vengono usati moduli ES).

3. Persistenza dei dati e schema del database

`db.js` crea all'avvio la cartella `data/` e le tre tabelle con `CREATE TABLE IF NOT EXISTS`, quindi l'avvio si può ripetere senza perdere dati. Lo schema, in forma ridotta:

```
projects (
  id INTEGER PRIMARY KEY AUTOINCREMENT,  name TEXT NOT NULL,  description TEXT,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,  updated_at DATETIME DEFAULT CURRENT_TIMESTAMP )

conversations (
  id ...,  project_id INTEGER NOT NULL,  topic TEXT NOT NULL,  created_at ...,  updated_at ...,
  FOREIGN KEY (project_id) REFERENCES projects(id) ON DELETE CASCADE )

messages (
  id ...,  conversation_id INTEGER NOT NULL,  content TEXT NOT NULL,  created_at ...,
  role TEXT NOT NULL CHECK(role IN ('user', 'assistant')),
  FOREIGN KEY (conversation_id) REFERENCES conversations(id) ON DELETE CASCADE )
```

In SQLite il controllo delle chiavi esterne è disattivato di default e va abilitato su ogni connessione con `PRAGMA foreign_keys = ON`; senza questa istruzione la clausola `ON DELETE CASCADE` verrebbe ignorata e la cancellazione di un progetto lascerebbe conversazioni e messaggi orfani. Con la cascata un solo `DELETE` elimina tutto ciò che dipende dal progetto. Il vincolo `CHECK` sul ruolo fa sì che anche la risposta dell'assistente sia una normale riga di `messages`, distinta solo dal ruolo. Le date sono assegnate dal database con `CURRENT_TIMESTAMP`, in UTC e senza indicazione del fuso, perciò il client le converte esplicitamente prima di mostrarle. Il modulo `sqlite3` è asincrono, e le istruzioni di inizializzazione sono racchiuse in `db.serialize()` perché vengano eseguite nell'ordine scritto. Per ricreare il database da zero basta fermare il server, cancellare `data/database.sqlite` e riavviare.

4. Progettazione delle API REST

Gli endpoint ruotano attorno alle tre risorse: il path indica cosa si tratta e il metodo HTTP cosa farne (`GET` legge, `POST` crea, `DELETE` elimina). Le risorse figlie sono annidate nel path del padre dove serve il contesto, mentre le operazioni su una singola conversazione usano `/api/conversations/:id`, perché l'identificativo è già univoco. Tutte le risposte, errori compresi, sono in JSON.

Metodo e path	Dati ricevuti	Risposta	Errori gestiti
`GET /api/projects`	nessuno	200, array di progetti	500
`POST /api/projects`	body {name, description}	201, progetto creato	400 nome vuoto o non testuale, descrizione non testuale; 500
`DELETE /api/projects/:id`	id nel path	200, {message, id}	404; 500
`GET /api/projects/:projectId/conversations`	projectId nel path	200, array di conversazioni	404 progetto assente; 500
`POST /api/projects/:projectId/conversations`	body {topic}	201, conversazione creata	400 topic vuoto; 404 progetto assente; 500
`DELETE /api/conversations/:id`	id nel path	200, {message, id}	404; 500
`GET /api/conversations/:id/messages`	id nel path	200, array di messaggi	404 conversazione assente; 500
`POST /api/conversations/:id/messages`	body {content}	201, {userMessage, assistantMessage}	400 testo vuoto; 404 conversazione assente; 500
`GET /api/search?q=`	query string q	200, array di risultati	500
`GET /api/conversations/:id/export`	id nel path	200, oggetto JSON	404; 500

Ogni richiesta è validata prima di toccare il database. Nome, topic e contenuto devono essere stringhe non vuote dopo il `trim()`; il controllo del tipo, oltre a quello sul contenuto, evita che un valore numerico o un array faccia lanciare un'eccezione al server. Una risorsa inesistente dà sempre 404, anche nelle letture: per le conversazioni di un progetto e per i messaggi di una conversazione il server verifica prima l'esistenza del genitore, perché un array vuoto significherebbe "nessun elemento" e non "risorsa assente". Le creazioni rispondono 201 e restituiscono la risorsa con l'identificativo assegnato, letto da `this.lastID` nella callback di `db.run` (per questo la callback è una `function` e non una arrow function, che non espone le informazioni dell'istruzione). Con `this.changes` il server distingue una `DELETE` che ha rimosso una riga da una su una riga inesistente.

Gli errori che nascono fuori dai gestori sono resi in JSON da due middleware in fondo alla catena: un percorso `/api` sconosciuto dà 404, e un corpo con JSON malformato, intercettato da `express.json()`, dà 400. Le eliminazioni sono previste per progetti e conversazioni, oltre il minimo richiesto; la modifica (PUT/PATCH) non è stata implementata. Nell'invio di un messaggio il server salva prima quello dell'utente, poi genera e salva la risposta, e restituisce entrambi in `{ userMessage, assistantMessage }`. Una sola richiesta evita al client di coordinare due attese. I due inserimenti non sono in una transazione. Alla lettura di una conversazione vuota il server inserisce il messaggio di benvenuto dell'assistente, e i messaggi sono ordinati per `created_at, id`: la data ha risoluzione di un secondo, e l'id risolve gli ex aequo.

Esempio di invio di un messaggio e di errore di validazione:

```
POST /api/conversations/3/messages
{ "content": "ciao" }

201 Created
{ "userMessage":      { "id": 12, "role": "user", "content": "ciao", ... },
  "assistantMessage": { "id": 13, "role": "assistant", "content": "Ciao! Sono ...", ... } }

POST /api/conversations/3/messages   { "content": "   " }
400 Bad Request
{ "error": "Il contenuto del messaggio non può essere vuoto." }
```

5. Front-end e gestione del DOM

L'interfaccia è una single page application con una sidebar, per navigare tra progetti e conversazioni, e un'area chat. `app.js` tiene un oggetto `state` con progetto e conversazione selezionati; ogni interazione lo aggiorna, chiama l'API con `fetch` e ricostruisce la parte di DOM interessata. Con `async/await` il codice si legge in ordine sequenziale senza bloccare il browser. `fetch` rifiuta la Promise solo per errori di rete: una risposta 400 o 404 è per lui un successo, perciò ogni chiamata controlla `res.ok`.

All'invio di un messaggio il form intercetta l'evento `submit` (scatta con il pulsante e con Invio) e chiama `preventDefault()`, altrimenti il browser ricaricherebbe la pagina. Il campo viene svuotato e i controlli disabilitati per evitare invii doppi, poi parte la `POST`. Alla risposta i due messaggi sono accodati alla chat con `appendMessage()`, senza riscaricare la cronologia, e lo scroll scende in fondo; nel blocco `finally` i controlli tornano attivi. Se l'invio fallisce il testo viene rimesso nel campo, così l'utente non deve riscriverlo.

Il feedback sull'esito delle operazioni è dato da notifiche temporanee, sia per i successi sia per gli errori. Per gli errori restituiti dal server si mostra il messaggio del campo `error` della risposta JSON, con un testo generico di riserva; un testo apposito segnala che il server non è raggiungibile. Anche i dati mancanti (nome o topic vuoti, messaggio vuoto, ricerca vuota) sono segnalati senza chiamare il server. Il testo delle notifiche è inserito con `textContent`, mentre quello di messaggi, topic e nomi passa da `escapeHtml()` prima di `innerHTML`: senza questo passaggio un tag `<script>` scritto in un messaggio, salvato nel database, verrebbe eseguito a ogni apertura della conversazione (XSS memorizzato).

Le date dei messaggi riletti dal database, in formato `YYYY-MM-DD HH:MM:SS` e in UTC, sono convertite aggiungendo `T` e `Z` prima di creare l'oggetto `Date`: senza la conversione il browser le leggerebbe come ora locale e mostrerebbe un orario sfasato rispetto ai messaggi appena inviati. La sidebar è costruita in un `DocumentFragment` inserito nel DOM una sola volta, e i gestori di click sono collegati dopo la creazione degli elementi. Dopo una creazione o un'eliminazione la sidebar è ricaricata dal server, perché l'elenco è breve e la vista deve coincidere con il database; la cronologia invece, potenzialmente lunga, viene solo estesa.

6. Logica di simulazione delle risposte

Il server non dipende da API esterne: la funzione `generateAssistantResponse(userPrompt, conversationTopic)` porta il prompt in minuscolo e confronta il testo con alcune parole chiave, in un'unica catena di `if / else if`. Le risposte sono template literal che inseriscono il topic e, in due casi su quattro, anche il prompt, quindi lo stesso codice produce testi diversi a seconda del contesto (punto 4.4).

```
function generateAssistantResponse(userPrompt, conversationTopic) {
  const promptLower = userPrompt.toLowerCase();
  if (promptLower.includes('ciao') || promptLower.includes('salve'))
      return `Ciao! Sono il tuo assistente per il topic "${conversationTopic}". ...`;
  else if (promptLower.includes('spiega') || promptLower.includes('cosa'))
      return `Riguardo a "${userPrompt}", ecco una spiegazione ... "${conversationTopic}": ...`;
  else if (promptLower.includes('grazie'))
      return `Prego, ... Se hai altre domande su "${conversationTopic}", ...`;
  else
      return `Ho ricevuto la tua richiesta su "${userPrompt}" e la sto elaborando. ...`;
}
```

L'ordine dei rami conta: un prompt come "ciao, puoi spiegare..." soddisfa due condizioni ma è gestito dal primo ramo. Il messaggio di benvenuto di una conversazione vuota è prodotto dalla stessa funzione, con il prompt "ciao". La funzione è volutamente semplice: non imita un modello linguistico, ma mostra che il server elabora l'input.

7. Funzionalità aggiuntive

Ricerca nello storico. Il client invia il testo a `GET /api/search?q=` dopo averlo passato a `encodeURIComponent`, perché spazi e caratteri speciali non rompano l'URL. Il server racchiude il termine tra `%`, il jolly di `LIKE`, e lo confronta con il topic, il nome del progetto e il contenuto dei messaggi. La query unisce le tabelle con una `JOIN` tra conversazioni e progetti e una `LEFT JOIN` sui messaggi: la seconda serve a trovare anche le conversazioni ancora senza messaggi, che una `JOIN` normale escluderebbe pur corrispondendo per topic o progetto. I risultati sono mostrati in una finestra modale con topic, progetto ed eventuale messaggio trovato; un click apre la conversazione.

Esportazione JSON. Il server legge conversazione, nome del progetto e messaggi in ordine cronologico e risponde con un oggetto che contiene `project`, `topic`, `exported_at`, `total_messages` e l'elenco `messages` con ruolo, testo e data. JSON conserva il ruolo di ogni messaggio e può essere riletto da altri programmi. Il download avviene nel client: la risposta è serializzata in un `Blob`, `URL.createObjectURL` ne crea un indirizzo temporaneo e un link con attributo `download` viene cliccato da codice; l'indirizzo è poi rilasciato con `revokeObjectURL`.

8. Stile e layout (CSS)

La grafica adotta un tema scuro. Il layout usa flexbox: il contenitore principale è una riga con la chat che occupa lo spazio residuo (`flex-grow: 1`) e la `.sidebar` di larghezza fissa, che la proprietà `order` porta a destra senza cambiare l'ordine logico dell'HTML. La larghezza è la variabile `--sidebar-width`, modificata da JavaScript, e una `transition` produce l'animazione di apertura e chiusura; il contenuto interno ha larghezza fissa, così il testo scorre via senza andare a capo. L'area dei messaggi è una colonna con `overflow-y: auto`, quindi scorre solo la cronologia. Colori, bordi e testi sono variabili in `:root` (`--bg-main`, `--bg-sidebar`, `--text-primary`, `--text-muted`), modificabili in un solo punto. Le bolle usano le classi `.user-message` e `.assistant-message`, con `align-self` opposto: utente a destra, assistente a sinistra. I pulsanti non utilizzabili (invio, esportazione, eliminazione senza conversazione selezionata) sono disabilitati e attenuati.

9. Conclusioni e limitazioni

Il progetto ha permesso di mettere in pratica API REST, manipolazione del DOM e persistenza relazionale in Node.js ed SQLite. Le limitazioni note sono le seguenti. Non c'è autenticazione: il database è locale e condiviso. Le risposte seguono regole fisse e non un modello linguistico. La colonna `updated_at` è aggiornata per le conversazioni a ogni messaggio ma non per i progetti, che di fatto sono ordinati per data di creazione.